

=====

=====

EXAMEN FINAL — PROGRAMACIÓN AVANZADA EN C++

Universidad San Francisco de Quito — USFQ

Nombre: Damaris Yacelga

Fecha: 13/07/2026

Nota: _____ / 100

PARTE 0 — Opción múltiple SIN IDE 15 min (30 pts)

PARTE 1 — Análisis escrito SIN IDE 20 min (30 pts)

PARTE 2 — Implementación CON IDE 25 min (40 pts)

NOTA: Todos los fragmentos asumen:

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

=====

=====

|| PARTE 0 — OPCIÓN MÚLTIPLE (SIN IDE) 30 pts ||

|| 15 preguntas · 2 pts c/u · Una sola respuesta correcta ||

|| Escribe la letra en la línea "Respuesta: ____" ||

Q1. (2 pts) ¿Cuál es la diferencia entre `new int(5)`
y `new int[5]`?

- A) No hay diferencia — ambos reservan 5 enteros
- B) `new int(5)` reserva un entero con valor 5;
`new int[5]` reserva un arreglo de 5 enteros
- C) `new int(5)` reserva 5 bytes; `new int[5]` reserva 5 enteros
- D) `new int[5]` inicializa todos los elementos en 5

Respuesta: B

Q2. (2 pts) ¿Qué ocurre si usas `delete` en lugar de `delete[]`
para liberar un arreglo dinámico?

- A) Nada — `delete` y `delete[]` son equivalentes en la práctica
- B) Error de compilación
- C) Comportamiento indefinido — puede producir memory leak
o corrupción de memoria
- D) Solo libera el primer elemento del arreglo

Respuesta: C

Q3. (2 pts) ¿Qué ocurre si no llamas `close()` explícitamente en un archivo abierto con `ofstream`?

- A) El archivo queda corrupto y pierde todo su contenido
- B) Error de compilación — `close()` es obligatorio
- C) El archivo se cierra automáticamente cuando el objeto sale de scope — el destructor lo cierra
- D) El archivo queda bloqueado y no puede abrirse de nuevo

Respuesta: C

Q4. (2 pts) ¿Cuál es la diferencia entre leer con `>>` y con `getline()`?

- A) No hay diferencia — ambos leen una línea completa
- B) `>>` lee hasta el primer espacio en blanco; `getline()` lee toda la línea incluyendo espacios
- C) `getline()` solo funciona con archivos, no con `cin`
- D) `>>` lee toda la línea; `getline()` lee una palabra

Respuesta: B

Q5. (2 pts) ¿Por qué los atributos de una clase deben ser `private` y no `public`?

- A) Porque el compilador lo exige para que la clase compile
- B) Para controlar el acceso y garantizar que los datos solo se modifiquen de forma válida a través de métodos
- C) Porque los atributos public no pueden usarse en métodos de la misma clase
- D) Para que las clases hijas puedan acceder a ellos

Respuesta: B

Q6. (2 pts) ¿Qué hace la lista de inicialización en un constructor?

- A) Llama al constructor de la clase base únicamente
- B) Inicializa los atributos antes de que se ejecute el cuerpo del constructor
- C) Es solo una notación alternativa — es idéntico a asignar en el cuerpo del constructor
- D) Solo funciona con atributos de tipo primitivo

Respuesta: B

Q7. (2 pts) ¿En qué orden se llaman constructores y destructores para este código?

```
class A {  
public:
```

```
A() { cout << "A"; }  
~A() { cout << "~A"; }  
};  
class B : public A {  
public:  
    B() { cout << "B"; }  
    ~B() { cout << "~B"; }  
};  
int main() { B obj; return 0; }
```

- A) A B ~A ~B
- B) B A ~B ~A
- C) A B ~B ~A
- D) B A ~A ~B

Respuesta: A

Q8. (2 pts) ¿Cuál es la diferencia entre atributos
private y protected en una clase base?

- A) No hay diferencia — ambos son inaccesibles desde fuera
- B) private: inaccesible desde clases hijas;
protected: accesible desde clases hijas pero no
desde fuera de la jerarquía
- C) protected es idéntico a public para las clases hijas
- D) private permite acceso desde clases hijas en C++ moderno

Respuesta: B

Q9. (2 pts) ¿Qué es el slicing en C++?

- A) Dividir un arreglo en dos partes iguales
- B) Liberar memoria parcialmente con delete
- C) La pérdida de la parte derivada de un objeto cuando se copia por valor a una variable de tipo base
- D) Acceder a un índice fuera de los límites de un arreglo

Respuesta: C

Q10. (2 pts) ¿Para qué sirve virtual en el destructor de una clase base?

- A) Para que el destructor se llame dos veces al liberar
- B) Para que al hacer delete por puntero base se llame el destructor correcto de la clase derivada
- C) Para evitar que las clases derivadas definan su propio destructor
- D) Es obligatorio siempre que la clase tenga atributos

Respuesta: B

Q11. (2 pts) ¿Qué hace override en C++?

- A) Permite que un método privado sea accesible desde fuera
- B) Obliga al compilador a verificar que el método existe como virtual en la clase base — error si no existe
- C) Crea una copia del método de la clase base
- D) Es equivalente a escribir virtual en la clase derivada

Respuesta: B

Q12. (2 pts) ¿Qué retorna `dynamic_cast<Perro*>(ptr)` si `ptr` apunta a un objeto `Gato`?

- A) Un puntero válido a `Perro` — el cast siempre funciona entre clases de la misma jerarquía
- B) `nullptr` — el tipo real no es `Perro`
- C) Un puntero a `void`
- D) Error de compilación

Respuesta: B

Q13. (2 pts) ¿Por qué `operator<<` para `cout` no puede ser un método de tu clase?

- A) Porque `<<` es un operador reservado que no puede sobrecargarse
- B) Porque `cout` (`ostream`) está a la izquierda — si fuera

método de tu clase la llamada quedaría al revés:

`obj << cout`

- C) Porque `operator<<` siempre debe retornar `void`
- D) Porque solo los operadores aritméticos pueden ser métodos de una clase

Respuesta: B

Q14. (2 pts) ¿Cuál es la diferencia entre `operator+` y `operator+=` en términos de lo que retornan?

- A) Ambos deben retornar el mismo tipo por referencia
- B) `operator+` retorna un objeto nuevo por valor;
`operator+=` retorna `*this` por referencia
- C) `operator+` retorna `*this`; `operator+=` retorna un objeto nuevo
- D) No hay diferencia en el tipo de retorno

Respuesta: B

Q15. (2 pts) ¿Qué es un dangling pointer y cómo se evita?

- A) Un puntero que apunta a `nullptr` — se evita inicializándolo
- B) Un puntero que apunta a memoria ya liberada — se evita asignando `nullptr` después de cada `delete`
- C) Un puntero declarado sin tipo — error de compilación

D) Un puntero que nunca se inicializa — se evita con new

Respuesta: B

PARTE 1 — ANÁLISIS ESCRITO	(SIN IDE) 30 pts	
5 preguntas · 6 pts c/u · Cierra la computadora ahora		

P1. MEMORIA DINÁMICA (6 pts)

¿Qué imprime este programa? ¿Cuántos errores tiene y cuáles?
Identifícalos todos y explica cómo corregir cada uno.

```
int main() {  
    int* a = new int(5);  
    int* b = a;  
    *b = 10;  
    delete a;  
    delete b;  
    cout << *a << endl;  
    return 0;  
}
```

¿Qué ocurre al ejecutar? ¿Llega a imprimir algo?

No, el programa tiene comportamiento indefinido puede terminar con un error antes de imprimir cualquier valor.

Errores (lista todos):

1. Doble delete
2. Se accede a memoria ya liberada (cout << *a)
3. Después de liberar la memoria los punteros apuntando a una dirección inválida, dangling pointers

Corrección:

```
Int main()
```

```
int* a = new int(5);
```

```
int* b = a;
```

```
*b = 10;
```

```
delete a;
```

```
a = nullptr;
```

```
b = nullptr;
```

P2. ARCHIVOS DE TEXTO (6 pts)

¿Qué queda escrito en el archivo notas.txt después de ejecutar este programa DOS veces seguidas?

¿Qué imprime en consola la segunda ejecución?

```
int main() {  
    ofstream f("notas.txt", ios::app);  
    f << "Examen: 18" << endl;  
    f.close();  
}
```

```
Ofstream g("notas.txt");  
string linea;  
while (getline(g, linea))  
    cout << linea << endl;  
g.close();  
return 0;  
}
```

Contenido de notas.txt después de la 2da ejecución:

Examen: 18

Examen: 18

Consola en la 2da ejecución:

Examen: 18

Examen: 18

¿Qué buena práctica falta en este código?

Verificar que los archivos se abrieron correctamente antes de leer o escribir.

P3. ENCAPSULAMIENTO (6 pts)

¿Qué imprime este programa? Luego responde las preguntas.

```
class Sensor {  
    double lectura;  
    string unidad;  
public:  
    Sensor(double l, string u) : lectura(l), unidad(u) {
```

```

        cout << "Sensor(" << lectura << ")" << endl;
    }
    ~Sensor() {
        cout << "~Sensor(" << lectura << ")" << endl;
    }
    void setLectura(double l) {
        if (l >= 0) lectura = l;
    }
    double getLectura() const { return lectura; }
    void imprimir() const {
        cout << lectura << " " << unidad << endl;
    }
};

```

```

void calibrar(Sensor s) {
    s.setLectura(0.0);
    s.imprimir();
}

```

```

int main() {
    Sensor s1(36.6, "C");
    calibrar(s1);
    s1.imprimir();
    {
        Sensor s3(0.5, "V");
    }
    return 0;
}

```

Salida (en orden):

Sensor(36.6)
0 C
~Sensor(0)
36.6 C
Sensor(0.5)
~Sensor(0.5)
~Sensor(36.6)

¿Por qué calibrar(s1) no modifica el objeto original s1?

Porque el parámetro Sensor s se pasa por valor, por lo que se crea una copia del objeto los cambios afectan únicamente a la copia y no al objeto original.

P4. HERENCIA (6 pts)

¿Qué imprime este programa? ¿Qué fenómeno ocurre en la línea marcada con (★)?

```
class Vehiculo {  
protected:  
    string marca;  
    int anio;  
public:  
    Vehiculo(string m, int a) : marca(m), anio(a) {  
        cout << "Vehiculo(" << marca << ")" << endl;  
    }  
    ~Vehiculo() {  
        cout << "~Vehiculo(" << marca << ")" << endl;  
    }  
};
```

```

void describir() {
    cout << marca << " " << anio << endl;
}
};

class Moto : public Vehiculo {
    int cilindrada;
public:
    Moto(string m, int a, int c)
        : Vehiculo(m, a), cilindrada(c) {
        cout << "Moto(" << cilindrada << "cc)" << endl;
    }
    ~Moto() {
        cout << "~Moto(" << cilindrada << "cc)" << endl;
    }
    void describir() {
        cout << marca << " " << anio
            << " " << cilindrada << "cc" << endl;
    }
};

```

```

int main() {
    Moto m("Honda", 2024, 600);
    Vehiculo v = m;        // (★)
    v.describir();
    m.describir();
    return 0;
}

```

Salida:

Vehiculo(Honda)

Moto(600cc)

Honda 2024

Honda 2024 600cc

~Vehiculo(Honda)

~Moto(600cc)

~Vehiculo(Honda)

Fenómeno en (★): object slicing

Explicación: Vehiculo v = m, se copia únicamente la parte correspondiente a Vehiculo.

P5. POLIMORFISMO (6 pts)

¿Qué imprime este programa? Identifica el error que tiene y explica qué consecuencia produce en tiempo de ejecución.

```
class Figura {  
public:  
    virtual double area() = 0;  
    virtual void imprimir() {  
        cout << "Area: " << area() << endl;  
    }  
    ~Figura() { cout << "~Figura" << endl; }  
};
```

```
class Circulo : public Figura {  
    double radio;  
public:
```

```
Circulo(double r) : radio(r) {}  
double area() override { return 3.14 * radio * radio; }  
~Circulo() { cout << "~Circulo" << endl; }  
};
```

```
class Cuadrado : public Figura {  
    double lado;  
public:  
    Cuadrado(double l) : lado(l) {}  
    double area() override { return lado * lado; }  
    ~Cuadrado() { cout << "~Cuadrado" << endl; }  
};
```

```
int main() {  
    Figura* formas[2];  
    formas[0] = new Circulo(3.0);  
    formas[1] = new Cuadrado(4.0);  
    for (int i = 0; i < 2; i++) {  
        formas[i]→imprimir();  
        delete formas[i];  
        formas[i] = nullptr;  
    }  
    return 0;  
}
```

Salida:

Area: 28.26

~Figura

Area: 16

~Figura

Error: El destructor de Figura no es virtual.

Consecuencia: solo se ejecuta ~Figura() y no el destructor de Circulo o Cuadrado, lo que puede provocar liberación incompleta de recursos.

Corrección:

```
virtual ~Figura() {  
    cout << "~Figura" << endl;  
}
```

PARTE 2 — IMPLEMENTACIÓN	(CON IDE) 40 pts
Abre el IDE cuando el profesor lo indique	

CONTEXTO:

Estás construyendo el sistema de una biblioteca digital.

La biblioteca tiene distintos tipos de publicaciones.

Todas comparten información básica, pero cada tipo tiene comportamiento específico al mostrarse y calcular su duración de préstamo.

LO QUE DEBES IMPLEMENTAR:

1. Clase base abstracta Publicacion:

Atributos (protected):

string titulo

string autor

int anio

Constructor: Publicacion(string titulo,
string autor, int anio)

virtual int diasPrestamo() const = 0

virtual void imprimir() const

→ imprime: [anio] titulo — autor

Prestamo: diasPrestamo() dias

string getTitulo() const → retorna titulo

virtual ~Publicacion() {}

Nota: para imprimir con cout << obj, define FUERA de la clase:

ostream& operator<<(ostream& os, const Publicacion& p)

→ llama p.imprimir() y retorna os

2. Clase Libro (hija de Publicacion):

Atributo (private): int paginas

Constructor: Libro(string titulo, string autor,
int anio, int paginas)

int diasPrestamo() const override

→ retorna 14 si paginas <= 300, si no retorna 21

void imprimir() const override

→ llama Publicacion::imprimir()

→ imprime: Paginas: paginas

3. Clase Revista (hija de Publicacion):

Atributo (private): int numero

Constructor: Revista(string titulo, string autor,
int anio, int numero)

int diasPrestamo() const override → retorna 7

void imprimir() const override

→ llama Publicacion::imprimir()

→ imprime: Numero: numero

4. Main:

- Crea un arreglo Publicacion* de 4 elementos
- Asigna con new: 2 Libros y 2 Revistas
- Recorre con un bucle e imprime cada publicación usando operator<<
- Escribe en "catalogo.txt" el titulo y los dias de préstamo de cada publicación (una línea c/u)
- Libera la memoria con delete + nullptr

SALIDA ESPERADA EN CONSOLA (con tus propios datos):

[2021] Don Quijote — Cervantes

Prestamo: 21 días

Paginas: 863

[2023] National Geographic — Varios

Prestamo: 7 días

Numero: 312

... (los 4 objetos)

CRITERIOS DE EVALUACIÓN (40 pts):

- [] (8 pts) Clase Publicacion: abstracta correcta,
imprimir() const, getTitulo() const,
operator<< como función libre fuera de la clase,
virtual ~Publicacion() {}
- [] (12 pts) Libro y Revista: constructores correctos
llamando Publicacion(...) en lista de init,
diasPrestamo() con lógica correcta,
imprimir() con llamada a Publicacion::imprimir()
- [] (10 pts) Main: arreglo Publicacion*, new para cada
objeto, bucle polimórfico con operator<<,
delete + nullptr
- [] (6 pts) Escritura en catalogo.txt: titulo y dias
de préstamo, una línea por publicación,
verificación de apertura del archivo
- [] (4 pts) Programa compila sin errores y produce
la salida esperada

=====

=====

FIN DEL EXAMEN FINAL

=====

=====